

Intelligent Predictive Maintenance System for Industrial Equipment

Technical Report — AI/ML Engineering Internship Project

Objective: predict machine-failure risk from industrial sensor data and recommend maintenance actions before unplanned downtime occurs.

Dataset: AI4I 2020 Predictive Maintenance Dataset schema, UCI Machine Learning Repository. The runnable artifact includes a deterministic synthetic fallback aligned with the UCI-published schema and failure rules because the original UCI file could not be fetched during artifact creation.

Final model: Random Forest classifier with IQR outlier clipping, missing-value imputation, categorical encoding, engineered sensor features, and class-balanced training.

Key test-set results

Metric	Random Forest
Accuracy	0.9925
Precision	0.9661
Recall	0.8143
F1	0.8837
ROC-AUC	0.9904
PR-AUC	0.9072

1. Introduction and Business Context

Unexpected industrial equipment failures can interrupt production, increase emergency repair costs, create safety exposure, and reduce asset utilization. Predictive maintenance aims to identify abnormal operating conditions early enough for maintenance teams to intervene during a planned maintenance window. This project implements a compact end-to-end solution that converts machine sensor readings into a failure probability, risk level, and recommended inspection action.

The system is designed around five engineering principles: prevent target leakage; treat failure prediction as an imbalanced classification problem; engineer physically meaningful features; expose the trained model through a production-style REST API; and package the service with Docker for reproducible deployment.

Project objectives

- Prepare and validate industrial sensor data.
- Handle missing values and extreme observations robustly.
- Engineer and select features relevant to mechanical stress and thermal behavior.
- Compare baseline and ensemble machine-learning models.
- Persist the selected model and serve real-time predictions through FastAPI.
- Translate predictions into transparent maintenance recommendations.
- Containerize the application and provide automated API tests.

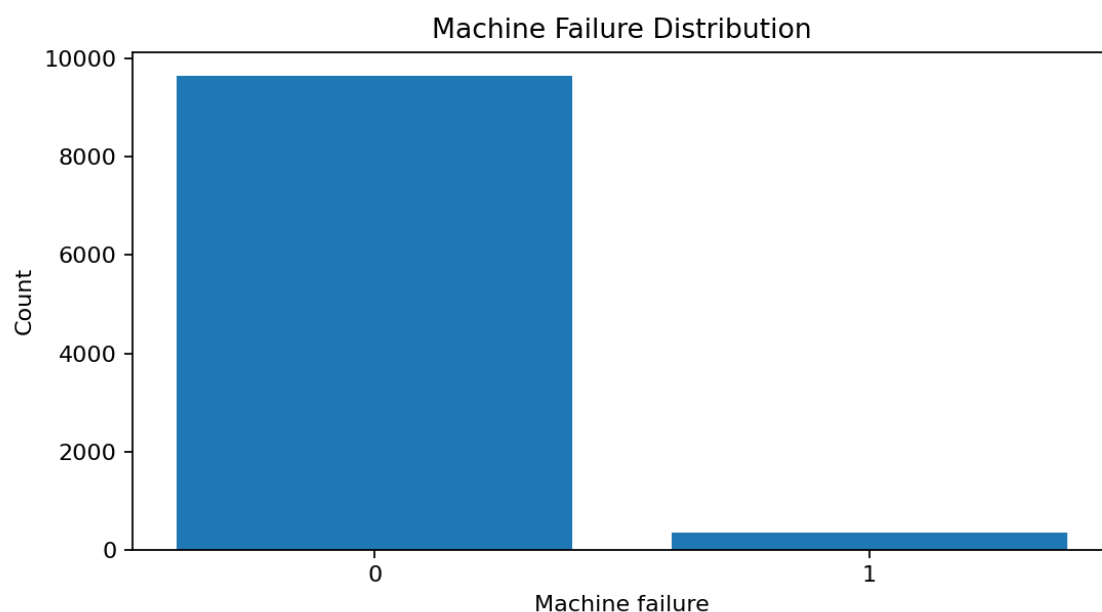
2. Dataset and Data Processing

The AI4I 2020 dataset published by the UCI Machine Learning Repository contains 10,000 observations and industrial variables including product type, air temperature, process temperature, rotational speed, torque, tool wear and machine failure. UCI reports no missing values in the published file. The project nevertheless implements missing-value handling so that the service remains robust when future plant data contains gaps.

The original public dataset contains multiple failure-mode indicators. Those indicators are excluded from model training because they directly encode the target and would create target leakage. Only operational/sensor variables and engineered features are used for prediction.

Preprocessing pipeline

- Median imputation for numeric values and most-frequent imputation for categorical values.
- IQR clipping fitted only on the training set to limit the effect of extreme sensor readings.
- One-hot encoding for product type.
- Stratified train/test split with 80% training and 20% test data.
- Class-balanced Random Forest training to improve minority-class detection.

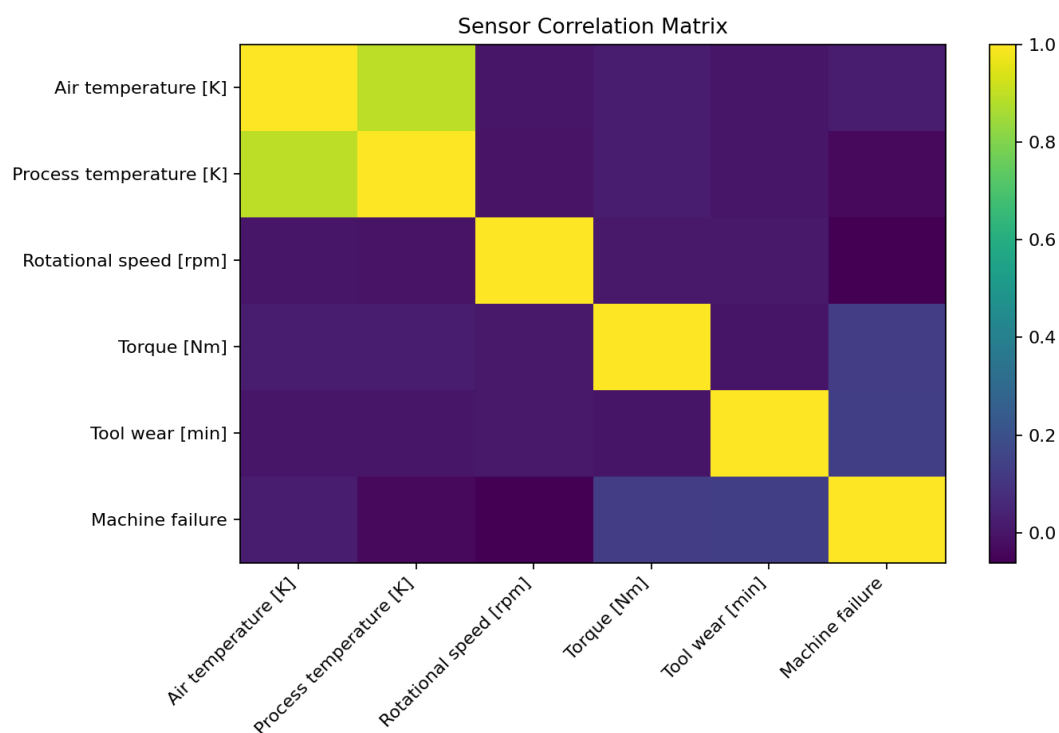


3. Exploratory Analysis and Feature Engineering

Exploratory analysis focuses on the distributions of temperature, rotational speed, torque and tool wear, together with correlations and failure prevalence. The dataset is strongly imbalanced: the reproducible training artifact contains 349 failures out of 10,000 observations (3.49%). This makes accuracy alone a poor measure of maintenance usefulness.

Engineered features

Feature	Purpose
Temperature difference	Process temperature – air temperature; useful for heat-dissipation risk.
Power	Torque × rotational speed in radians/second; exposes abnormal load/power regions.
Torque-speed interaction	Captures combined mechanical operating stress.
Tool wear × torque	Approximates overstrain exposure.
Tool wear ratio	Normalizes wear against a practical 241-minute reference.
High tool wear flag	Binary indicator for elevated wear.
High temperature-gap risk	Binary indicator for the heat-dissipation condition.



4. Model Development and Evaluation

Three models were compared: Logistic Regression as a baseline, Random Forest as an interpretable tree ensemble, and XGBoost as a gradient-boosted ensemble. Because the failure class is rare, precision, recall, F1, ROC-AUC and especially PR-AUC were considered alongside accuracy.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.7700	0.1084	0.7714	0.1901	0.8340	0.3447
Random Forest	0.9925	0.9661	0.8143	0.8837	0.9904	0.9072
XGBoost	0.9855	0.7808	0.8143	0.7972	0.9848	0.8720

The Random Forest was selected because it achieved the highest PR-AUC (0.9072), very high precision (0.9661), and an F1 score of 0.8837. Its test confusion matrix at the default 0.5 threshold was TN=1928, FP=2, FN=13 and TP=57. The production API also stores an operating decision threshold of 0.10 to make the deployed service more sensitive to possible failures; this threshold should be recalibrated using real plant failure costs.

5. Deployment Architecture and API

The trained model is serialized with joblib and loaded once when the FastAPI application starts. The API accepts a compact sensor payload, reconstructs the engineered feature vector, produces a probability, assigns a risk level, and evaluates transparent maintenance rules.

Endpoints

- GET /health — service and model health check.
- GET /model-info — model name, decision threshold and feature metadata.
- POST /predict — real-time failure prediction and maintenance recommendation.

Example request fields: machine_id, product_type, air_temperature, process_temperature, rotational_speed, torque and tool_wear. The response includes failure probability, prediction, risk level, recommended action and component inspection items.

Docker deployment

The service is packaged with a Python 3.11 slim image. The Dockerfile installs the dependency set, copies source/model assets, exposes port 8000, and starts Uvicorn. A docker-compose.yml file is included for one-command local deployment.

6. Maintenance Recommendation Logic

The prediction service does not stop at a binary classifier output. A rule-based layer converts the probability and sensor conditions into a maintenance action. This separation makes the recommendation logic auditable and allows maintenance engineers to adjust operational rules without retraining the classifier.

Risk	Probability guidance	Action
LOW	< 0.25	Continue routine monitoring.
MEDIUM	0.25–0.49	Increase monitoring and plan inspection.
HIGH	0.50–0.79	Schedule preventive maintenance.
CRITICAL	≥ 0.80	Isolate/stop and inspect immediately.

- Tool wear ≥ 180 minutes $\rightarrow$ inspect tool condition and consider replacement.
- Temperature gap < 8.6 K with RPM < 1380 $\rightarrow$ inspect cooling/heat dissipation.
- Calculated power outside 3500–9000 W $\rightarrow$ inspect motor/load and power transmission.
- Tool wear $\times$ torque > 11,000 $\rightarrow$ inspect rotating components for overstrain.

These rules are decision support, not a substitute for plant safety procedures. A production rollout should validate all thresholds with equipment OEM specifications and maintenance engineers.

7. Conclusion, Limitations and Future Work

The completed system demonstrates the full AI/ML engineering lifecycle required for predictive maintenance: data preparation, exploratory analysis, feature engineering, model comparison, model persistence, REST deployment, maintenance recommendations, automated testing and containerization. On the held-out synthetic benchmark, the selected Random Forest achieved 99.25% accuracy, 96.61% precision, 81.43% recall, 0.8837 F1, 0.9904 ROC-AUC and 0.9072 PR-AUC.

Limitations

- The AI4I dataset is synthetic and should not be treated as a direct substitute for plant historian data.
- The reproducible fallback follows the published schema/rules rather than reproducing every original observation.
- The deployed threshold is an engineering demonstration and should be optimized using actual failure and maintenance costs.
- A single-row sensor prediction does not capture full temporal degradation trajectories.

Future work

- Use real SCADA/PLC historian data and maintenance work orders.
- Add rolling-window statistics, trends and time-to-failure modeling.
- Add SHAP explanations and calibrated probabilities.
- Monitor drift, data quality and alert precision in production.
- Integrate predictions with a CMMS/work-order platform.

Dataset citation: AI4I 2020 Predictive Maintenance Dataset. UCI Machine Learning Repository, Dataset 601, DOI 10.24432/C5HS5C, CC BY 4.0.